5. System Design

System Design translates the analytical models from Chapter 4 into technical blueprints. This chapter details the frontend component structure (React routing), the backend database schema (with a complete Entity-Relationship Diagram), the REST API contracts, and a 3-tier system architecture overview.

5.1 Front End Design

The frontend application is structured as a component-based Single-Page Application (SPA) using React 18, React Router v6, and Axios. The project folder structure enforces separation of concerns:

- **`/src/components`**: Reusable presentation components (e.g., `Navbar.jsx`, `ProductCard.jsx`).
- **`/src/pages`**: Container pages bound to application routes (e.g., `Home.jsx`, `Cart.jsx`, `VendorProducts.jsx`).
- **`/src/context`**: State providers (`AuthContext.jsx` for JWT login state, `CartContext.jsx` for cart updates).
- **`/src/services`**: Utility scripts, primarily `api.js` which contains the Axios instance with automatic JWT token interceptors.

React Page Routing Structure

URL Path	React Component	Required Role	Access Type
`/`	`Home.jsx`	None	Public
`/products`	`ProductList.jsx`	None	Public
`/products/:id`	`ProductDetail.jsx`	None	Public
`/login`	`Login.jsx`	None	Public
`/register`	`Register.jsx`	None	Public
`/cart`	`Cart.jsx`	None	Public
`/checkout`	`Checkout.jsx`	Customer	Protected (JWT)

`/orders`	`Orders.jsx`	Any Authenticated	Protected (JWT)
`/profile`	`Profile.jsx`	Any Authenticated	Protected (JWT)
`/vendors`	`Vendors.jsx`	None	Public
`/vendor`	`VendorDashboard.jsx`	Vendor	Protected (JWT)
`/vendor/products`	`VendorProducts.jsx`	Vendor	Protected (JWT)
`/vendor/orders`	`VendorOrders.jsx`	Vendor	Protected (JWT)
`/admin`	`AdminDashboard.jsx`	Admin	Protected (JWT)
`/admin/vendors`	`AdminVendors.jsx`	Admin	Protected (JWT)
`/admin/orders`	`AdminOrders.jsx`	Admin	Protected (JWT)
`/admin/products`	`AdminProducts.jsx`	Admin	Protected (JWT)
`/admin/users`	`AdminUsers.jsx`	Admin	Protected (JWT)
`/admin/customers`	`AdminCustomers.jsx`	Admin	Protected (JWT)

5.2 Back End Design — Entity-Relationship Diagram (ERD)

The backend database consists of six core relational tables. SQLite is used locally in development and swapped with PostgreSQL in production by changing a single Django `settings.py` configuration.

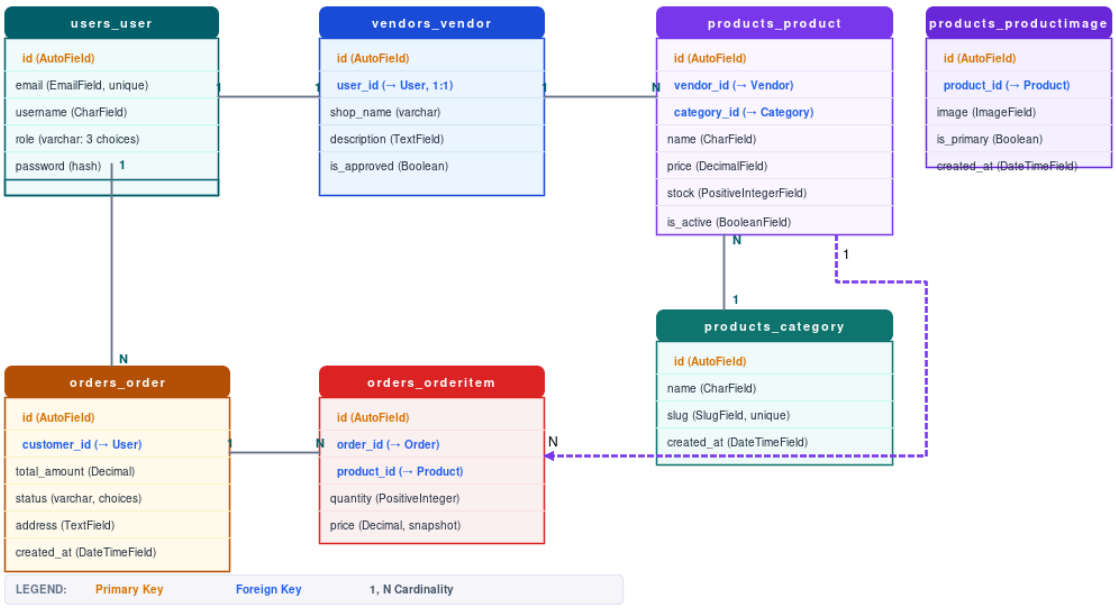

Figure 1 — Entity-Relationship Diagram (ERD) of the 6 database entities

5.3 Interface Design (REST API Contracts)

API endpoints serve as the communication interface between the React frontend client and the Django backend. Authentication is handled statelessly via JWT tokens sent in the `Authorization: Bearer <token>` header.

Method	API Endpoint	Auth Required	Request Payload	Response
PATCH	`/api/auth/me/`	Yes (Any role)	`{username, phone, address, shop_name?}`	200 — Updated profile
GET	`/api/vendors/`	No (Public)	None	200 — Approved vendor list
GET	`/api/products/`	No (Public)	Optional query params: `?search=&category=&vendor=`	200 — Product list

POST	`/api/products/`	Yes (Vendor)	`{name, price, stock, category_id, images[]}`	201 — Created product
PUT	`/api/products/{id}/`	Yes (Vendor Owner)	`{name, price, stock}`	200 — Updated product
DELETE	`/api/products/{id}/`	Yes (Vendor Owner)	None	204 — No Content
POST	`/api/orders/`	Yes (Customer)	`{items: [{product_id, quantity}], address}`	201 — Order receipt
GET	`/api/orders/`	Yes (Role-based)	None	200 — Filtered orders by role
PATCH	`/api/orders/{id}/update_status/`	Yes (Vendor / Admin)	`{status: "shipped"}`	200 — Updated status
PATCH	`/api/vendors/{id}/approve/`	Yes (Admin)	None	200 — Vendor approved
PATCH	`/api/vendors/{id}/reject/`	Yes (Admin)	None	200 — Vendor rejected
GET	`/api/dashboard/stats/`	Yes (Role-based)	None	200 — Role-specific stats & revenue

5.4 System Architecture Diagram

The platform uses a decoupled 3-tier architecture. The React SPA (client), Django REST API (application server), and SQLite/PostgreSQL (database) are fully independent, allowing each tier to be scaled separately.

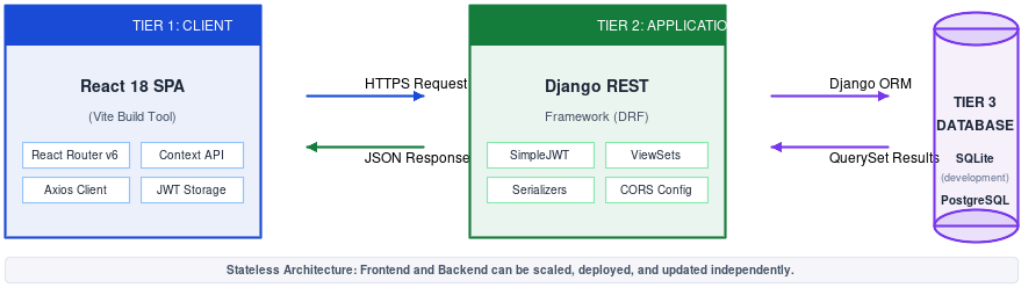

Figure 2 — 3-Tier Decoupled System Architecture layout